

Foundational Notes on Kalman Filtering in Reinforcement Learning

From Noisy Observations to Belief-Based Decision Making

March 30, 2026

Contents

1	Why This Combination Matters	2
2	A Running Example	2
3	The High-Level Logic	3
3.1	The Black-Box View	3
3.2	Why the Filter Helps the Policy	3
4	The Kalman Filter Side	3
4.1	Prediction and Correction	3
4.2	The Intuition	4
5	The Reinforcement Learning Side	4
5.1	What the Policy Sees	4
5.2	Reward Remains an RL Object	4
6	How the Two Pieces Fit Together	4
6.1	Pattern 1: Filter Before Policy	4
6.2	Pattern 2: Learn the Model, Then Filter	5
6.3	Pattern 3: Use Covariance for Exploration or Risk	5
7	A Concrete Numerical Walkthrough	5
7.1	Step 1: Predict	5
7.2	Step 2: Observe	6
7.3	Step 3: Correct	6
7.4	Step 4: Act	6
8	What This Buys You	6
9	What It Does Not Magically Solve	6
9.1	Linearity and Gaussian Assumptions	7
9.2	Model Error	7
9.3	Reward Does Not Become Easier	7
10	A Minimal Design Rule	7

1 Why This Combination Matters

Reinforcement learning is often introduced as if an agent could observe the world clearly, act, and then improve from experience. Real systems are rarely so generous. A robot sees a scene through noisy sensors. A trading system observes prices but not the hidden market regime. A wireless controller receives delayed and corrupted channel measurements. In each of these cases, the agent does not act on the true state of the world; it acts on incomplete evidence.

This is precisely where Kalman filtering becomes valuable. Its role is not to replace reinforcement learning, but to provide a disciplined estimate of the hidden state before the decision module commits to an action. Reinforcement learning answers the question “*what should be done next?*” Kalman filtering answers the prior question “*what is most likely happening now?*”

The combination is therefore natural:

- Kalman filtering reduces uncertainty in state estimation.
- Reinforcement learning uses that estimated state to optimize long-term reward.
- Together, they turn noisy measurements into more stable sequential decisions.

The central benefit is conceptual simplicity. One module estimates; the other decides. The engineering difficulty lies in making their assumptions compatible.

2 A Running Example

Consider a small ground robot moving along a straight corridor. Its true state at time t is

$$x_t = \begin{bmatrix} p_t \\ v_t \end{bmatrix},$$

where p_t is position and v_t is velocity.

The robot chooses an acceleration command a_t . Its motion is approximately

$$x_{t+1} = Ax_t + Ba_t + w_t,$$

where w_t is process noise. The camera does not reveal the full state. It only returns a noisy measurement of position,

$$y_t = Cx_t + r_t,$$

where r_t is measurement noise.

Suppose the robot is currently near the center of the corridor. The camera reports

$$y_t = 10.4 \text{ meters},$$

but the true position is unknown. The robot must decide whether to accelerate, brake, or hold speed. If it reacts directly to raw measurements, it may oscillate. If it first estimates the latent state, its actions become more coherent.

This simple corridor robot will be our guiding example throughout the note.

3 The High-Level Logic

3.1 The Black-Box View

The combined system can be understood as a four-stage pipeline:

1. The environment produces a hidden state x_t .
2. Sensors produce a noisy observation y_t .
3. A Kalman filter converts past information and the new observation into an estimate $\hat{x}_{t|t}$.
4. A reinforcement learning policy uses that estimate to choose an action a_t .

After the action is applied, the environment moves forward, reward is revealed, and the loop repeats.

The input to the filter is not merely the current sensor value. It also uses the previous estimate and the known control action. The output is therefore richer than a denoised measurement; it is a dynamically consistent belief about the underlying state.

3.2 Why the Filter Helps the Policy

Reinforcement learning suffers when its input representation is unstable. If two nearly identical physical situations produce very different raw observations, the policy must learn to average over unnecessary noise. A Kalman filter alleviates this in three ways:

- It smooths sensor noise by combining prediction and measurement.
- It reconstructs latent quantities such as velocity or bias that are not directly observed.
- It exposes uncertainty through the covariance matrix, which can inform risk-sensitive control or exploration.

In short, the filter gives the policy a more meaningful object to reason about than the raw sensor stream alone.

4 The Kalman Filter Side

4.1 Prediction and Correction

For the linear-Gaussian case, the Kalman filter alternates between prediction and correction:

Prediction. Use the previous estimate and the chosen action to forecast the next state.

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} + Ba_{t-1}.$$

At the same time, propagate the uncertainty forward.

Correction. Compare the actual measurement y_t with the predicted measurement $C\hat{x}_{t|t-1}$. The discrepancy is called the innovation. If the innovation is small, the model was already consistent with the new data. If it is large, the estimate should move toward the sensor.

The Kalman gain determines how strongly the correction reacts. A reliable sensor yields a large correction; an unreliable sensor yields a conservative one.

4.2 The Intuition

The filter is often described as an optimal fusion rule, but that phrase can feel abstract. A more concrete interpretation is this:

The predictor says where the system should be if the model were trustworthy.
The sensor says where it appears to be if the measurement were trustworthy.
The filter computes a principled compromise.

For the corridor robot, suppose the predicted position is 10.0 meters and the camera reports 10.4 meters. If the camera is noisy, the corrected estimate may move only to 10.1. If the camera is precise, the corrected estimate may move to 10.35. The estimate changes, but not naively.

5 The Reinforcement Learning Side

5.1 What the Policy Sees

In standard reinforcement learning, a policy maps state to action:

$$a_t \sim \pi(\cdot \mid s_t).$$

When the true state is not directly visible, a practical replacement is to feed the policy an estimated state:

$$a_t \sim \pi(\cdot \mid \hat{x}_{t|t}).$$

This substitution is more than a coding trick. It changes the learning problem from *acting on raw observations* to *acting on a belief about the hidden state*. That is the correct viewpoint whenever the world is only partially observed.

5.2 Reward Remains an RL Object

The filter estimates state, but it does not define what is desirable. That role belongs to reward design. For the robot, one might choose reward terms that:

- encourage progress toward a target location,
- penalize excessive acceleration,
- penalize collisions or leaving the corridor.

The policy then learns which actions maximize cumulative reward, but it learns from filtered state estimates rather than raw measurements.

6 How the Two Pieces Fit Together

6.1 Pattern 1: Filter Before Policy

This is the most basic and most useful architecture.

1. Use a Kalman filter to estimate the hidden state from actions and observations.
2. Feed the estimate into a policy or value function.

3. Train the reinforcement learning agent as usual.

This pattern is especially sensible when the dynamics are roughly linear and the primary difficulty is sensor noise.

6.2 Pattern 2: Learn the Model, Then Filter

In many applications, the matrices A , B , and C are not fully known. One may then learn an approximate dynamics model from data, and place a Kalman-style estimator on top of that learned model. This idea appears in model-based reinforcement learning and latent-state world models.

The advantage is adaptability. The danger is model mismatch. A filter built on a poor learned model may become confidently wrong.

6.3 Pattern 3: Use Covariance for Exploration or Risk

The covariance matrix summarizes uncertainty. An agent may exploit this information in at least two ways:

- explore more cautiously when state estimates are unreliable,
- choose robust actions when uncertainty is large.

This is where Kalman filtering contributes more than denoising. It provides a quantitative measure of ignorance.

7 A Concrete Numerical Walkthrough

Let the time step be 1 second. Suppose the motion model is

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}, \quad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Assume that after the previous update, the agent believes

$$\hat{x}_{t-1|t-1} = \begin{bmatrix} 8.0 \\ 1.0 \end{bmatrix}.$$

The agent chooses acceleration

$$a_{t-1} = 0.2.$$

7.1 Step 1: Predict

The predicted state is

$$\hat{x}_{t|t-1} = A\hat{x}_{t-1|t-1} + Ba_{t-1} = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 8.0 \\ 1.0 \end{bmatrix} + \begin{bmatrix} 0.5 \\ 1 \end{bmatrix} (0.2) = \begin{bmatrix} 9.1 \\ 1.2 \end{bmatrix}.$$

So before seeing the new camera frame, the robot predicts that it has moved to position 9.1 with velocity 1.2.

7.2 Step 2: Observe

The camera reports

$$y_t = 9.7.$$

The innovation is therefore

$$y_t - C\hat{x}_{t|t-1} = 9.7 - 9.1 = 0.6.$$

7.3 Step 3: Correct

Suppose the Kalman gain at this moment is

$$K_t = \begin{bmatrix} 0.6 \\ 0.2 \end{bmatrix}.$$

Then the corrected estimate becomes

$$\hat{x}_{t|t} = \hat{x}_{t|t-1} + K_t(0.6) = \begin{bmatrix} 9.1 \\ 1.2 \end{bmatrix} + \begin{bmatrix} 0.36 \\ 0.12 \end{bmatrix} = \begin{bmatrix} 9.46 \\ 1.32 \end{bmatrix}.$$

This single step already reveals the value of the method. The observation provided only position. The filter updated both position and velocity, because the dynamic model couples them.

7.4 Step 4: Act

The reinforcement learning policy now receives the filtered estimate $[9.46, 1.32]^\top$, not the raw observation 9.7. If the target is at position 12, the policy may decide to apply mild braking rather than aggressive acceleration, because it now believes the robot is already moving with positive speed.

Thus the filter shapes the policy input, and the policy determines the control action that will influence the next prediction step.

8 What This Buys You

The Kalman-RL combination is attractive for several practical reasons:

- It improves sample efficiency when raw observations are noisy.
- It produces smoother control signals.
- It handles partial observability in a principled way.
- It often reduces the burden on the policy network, since some inference is delegated to the estimator.

In engineering terms, it separates *state inference* from *decision optimization*, which is usually cleaner than forcing one large model to learn both from scratch.

9 What It Does Not Magically Solve

The method also has real limitations.

9.1 Linearity and Gaussian Assumptions

The classical Kalman filter is exact only for linear dynamics with Gaussian noise. Many reinforcement learning environments are neither. If the robot has contact dynamics, mode switches, or strongly nonlinear sensors, the classical filter may be too optimistic. In such cases one often moves to the extended Kalman filter, unscented Kalman filter, particle filter, or a learned latent-state model.

9.2 Model Error

If the transition model is wrong, the filter may confidently track the wrong latent state. Reinforcement learning then optimizes on top of a distorted internal picture. This can be more dangerous than simple noise, because the error is structured and persistent.

9.3 Reward Does Not Become Easier

Kalman filtering helps estimate the state; it does not specify the objective. A poorly designed reward still produces poor behavior. Estimation quality and control quality are related, but they are not the same problem.

10 A Minimal Design Rule

For a first implementation, the following rule is hard to improve upon:

If observations are noisy, the state is only partially observed, and the short-term physics are moderately structured, first build a state estimator, then train the reinforcement learning policy on top of that estimator.

This strategy is modest rather than fashionable, but in many real systems it is the correct starting point.

11 Final Perspective

Kalman filtering and reinforcement learning solve adjacent problems. The former estimates hidden state from uncertain evidence. The latter chooses actions that optimize long-term consequences. Their combination is powerful because good decisions require good beliefs.

One should therefore resist the temptation to view Kalman filtering as a minor preprocessing step. In partially observed control, the estimator is part of the intelligence of the system itself. A policy that sees the wrong world, however sophisticated, will act unwisely. A policy that sees a well-structured belief has a far better chance of learning something stable, data-efficient, and physically sensible.